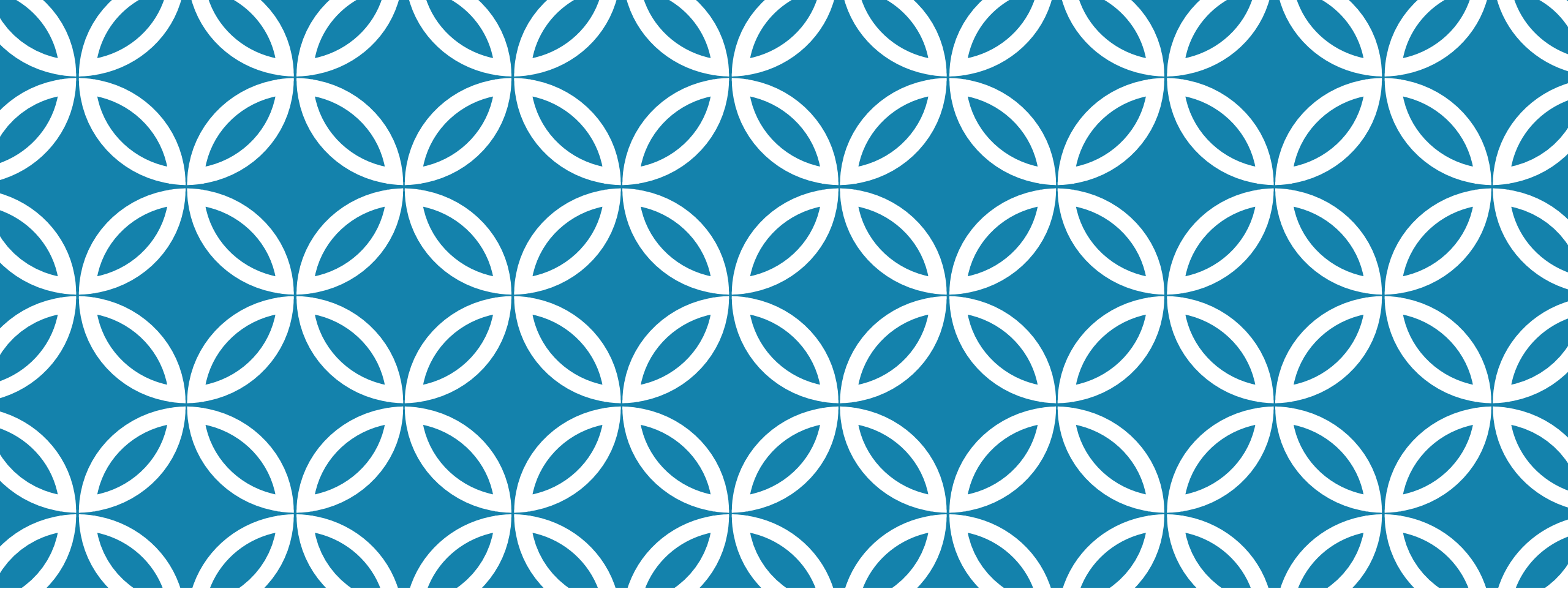




ARQUITETURA 80386

MECANISMO DE PROTEÇÃO

Prof. André Gustavo Adami

WHY PROTECTION?

“The purpose of the protection features of the 80386 is to help detect and identify bugs. The 80386 supports sophisticated applications that may consist of hundreds or thousands of program modules. In such applications, the question is how bugs can be found and eliminated as quickly as possible and how their damage can be tightly confined. To help debug applications faster and make them more robust in production, the 80386 contains mechanisms to verify memory accesses and instruction execution for conformance to protection criteria. These mechanisms may be used or ignored, according to system design objectives.”

Intel Corporation, **Intel 80386 Programmer's Reference Manual**. Santa Clara, CA, 1986.

PORQUE PROTEÇÃO?

“O propósito das características de proteção do 80386 é **ajudar a detectar e identificar bugs**. O 80386 suporta aplicações sofisticadas que podem consistir de centenas ou milhares de módulos de programas. Em tais aplicações, a questão é como **bugs podem ser encontrados e eliminados o mais rápido possível** e como os **seus danos podem ser rigorosamente confinados**. Para **ajudar a depurar aplicações mais rapidamente e torná-las mais robustas em produção**, o 80386 contém mecanismos para **verificar acessos à memória e execução de instruções** quanto à **conformidade a critérios de proteção**. Esses mecanismos podem ser usados ou ignorados, de acordo com os objetivos do projeto do sistema”.

Intel Corporation, **Intel 80386 Programmer's Reference Manual**. Santa Clara, CA, 1986.

PORQUE PROTEÇÃO?

Segmentação e paginação não são suficientes para impedir acesso de programas com problemas ou mal intencionados

- Um aplicativo poderia alterar o endereço base da GDT para uma área em que ele pudesse escrever

Foi implementada primeiramente no 80286 (mas não deu muito certo...)

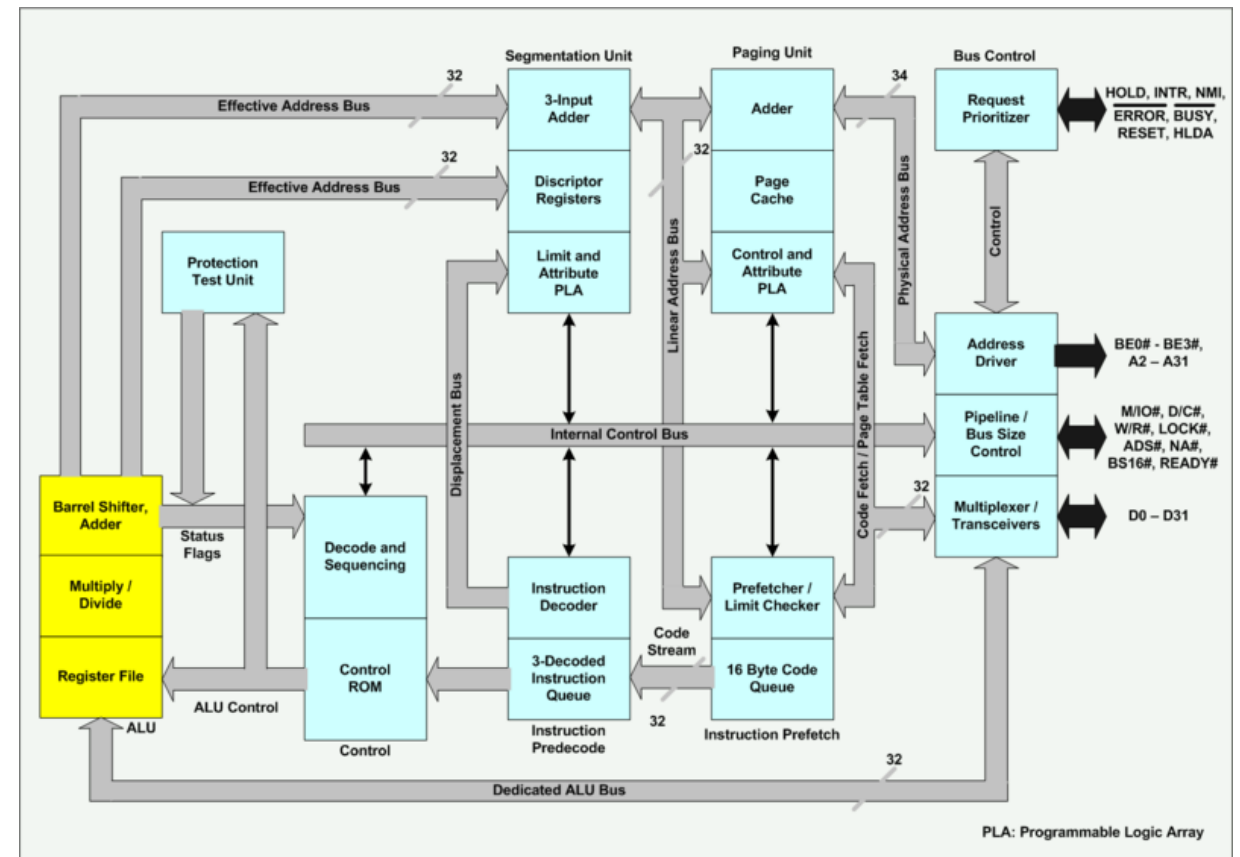
80386 — MECANISMO DE PROTEÇÃO

O hardware de proteção do 80386 é parte do hardware de gerenciamento de memória

- Proteção aplica-se a tradução de segmentos e de páginas

Aspectos de Proteção

- Checagem de
 - Tipos
 - Limites
- Restrição de
 - Endereços acessíveis
 - Pontos de entrada de procedimentos
 - Conjunto de instruções disponíveis



180386 — MECANISMO DE PROTEÇÃO

As verificações de proteção são realizadas automaticamente pelo processador quando o seletor de um descritor de segmento é carregado em um registrador de segmento

Cada referência a memória é verificada pelo hardware para checar se ela satisfaz os critérios de proteção

- Estas verificações ocorrem antes que o ciclo de memória comece. Assim, qualquer violação previne o começo do ciclo e uma exceção é gerada

Como as verificações são realizadas concorrentemente com a formação do endereço, o desempenho não é penalizado

180386 — MECANISMO DE PROTEÇÃO

O conceito de “privilégio” é fundamental para vários dos aspectos de proteção (restrição)

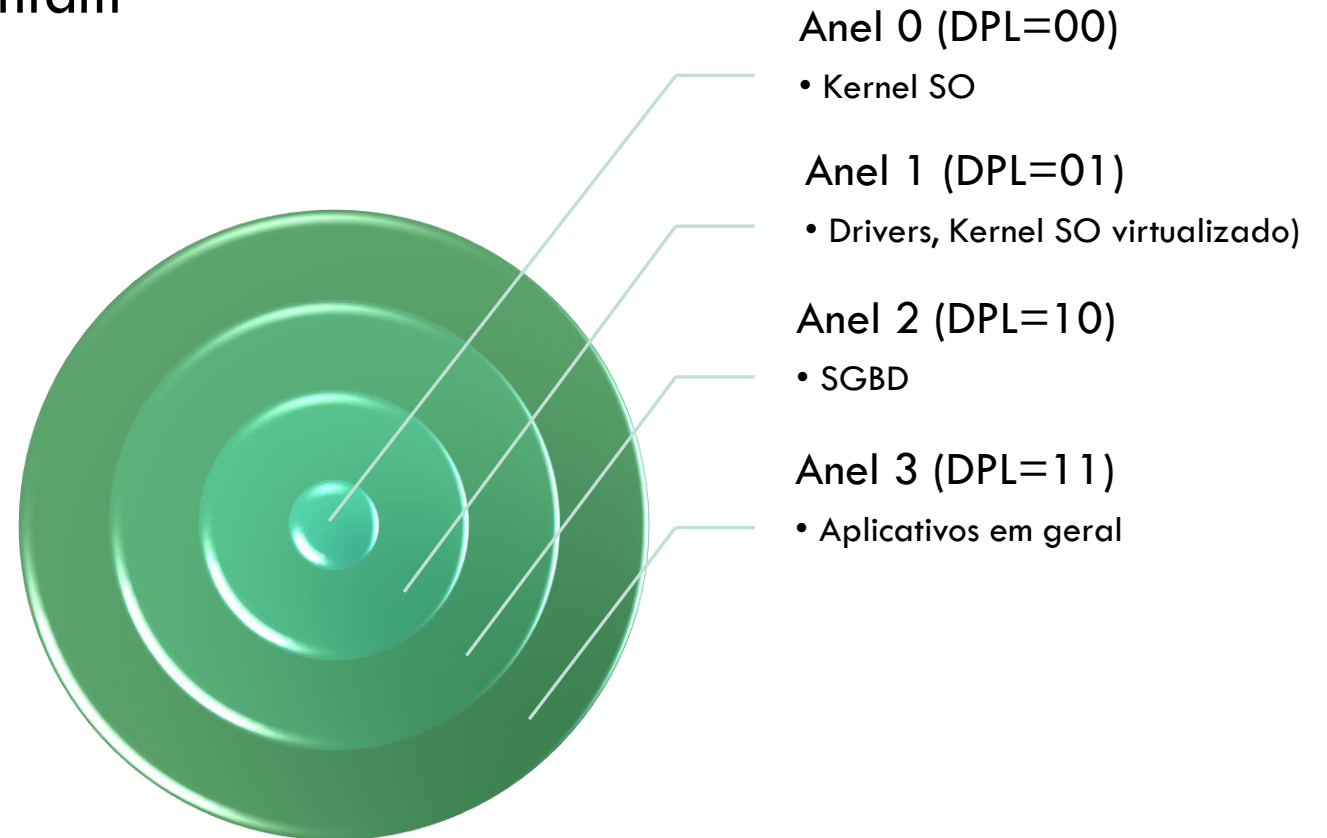
No caso de procedimentos, é o grau no qual o procedimento pode ser confiado de não fazer um erro que pode afetar outros procedimentos ou dados

No caso de dados, é o grau de proteção que uma estrutura de dados deveria ter para procedimentos menos confiáveis

80386 — NÍVEIS DE PRIVILÉGIO

O 80386 oferece 4 níveis de privilégio em ordem crescente de privilégio (3 é o de menor privilégio e 0 é o de maior)

- A maioria dos SO atuais implementam apenas 2 níveis: 0 e 3
- OS/2 implementava 3 níveis: 0, 2 (rotinas especiais que acessavam dispositivos), e 3



180386 — NÍVEIS DE PRIVILÉGIO

O processador reconhece níveis de privilégios nos seguintes elementos

- Descritores de segmento possuem o campo *Descriptor Privilege Level* (DPL)
- Seletores de segmento possuem o campo *Requestor's Privilege Level* (RPL)
 - Representa o nível de privilégio do procedimento que originou o seletor
- Nível de privilégio no seletor CS identifica o nível de privilégio do procedimento em execução (*Current Privilege Level* - CPL)
 - Normalmente o CPL é igual ao DPL do segmento que o processado está atualmente executando
 - Sistema Operacional trabalha com CPL=0, enquanto que aplicativos trabalham com CPL=3

180386 — NÍVEIS DE PRIVILÉGIO

O processador reconhece níveis de privilégios nos seguintes elementos

- Descritores de segmento possuem o campo *Descriptor Privilege Level* (DPL)
- Seletores de segmento possuem o campo *Requestor's Privilege Level* (RPL)
 - Representa o nível de privilégio do procedimento que originou o seletor
- Nível de privilégio no seletor CS identifica o nível de privilégio do procedimento em execução (*Current Privilege Level* - CPL)
 - Normalmente o CPL é igual ao DPL do segmento que o processado está atualmente executando
 - Sistema Operacional trabalha com CPL=0, enquanto que aplicativos trabalham com CPL=3

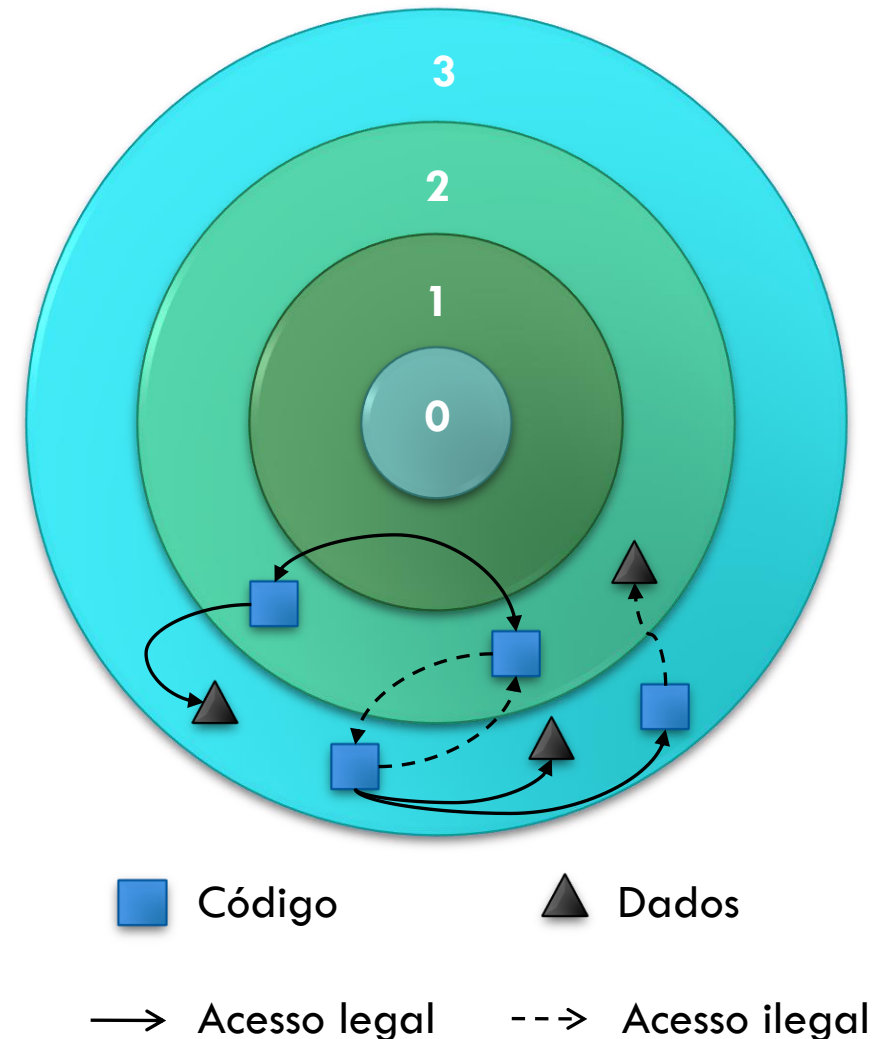
180386 — NÍVEIS DE PRIVILÉGIO

Um processo pode acessar somente dados de segmentos de menor ou igual nível (menor privilégio)

- Segmentos de dados com DPL=2 podem ser acessados por código somente com DPL=2, 1, ou 0

Para evitar problemas de código de má qualidade, os procedimentos não podem chamar procedimentos de menor privilégio

- Procedimento com CPL=1 pode chamar sub-rotinas em segmento de código somente com DPL=1



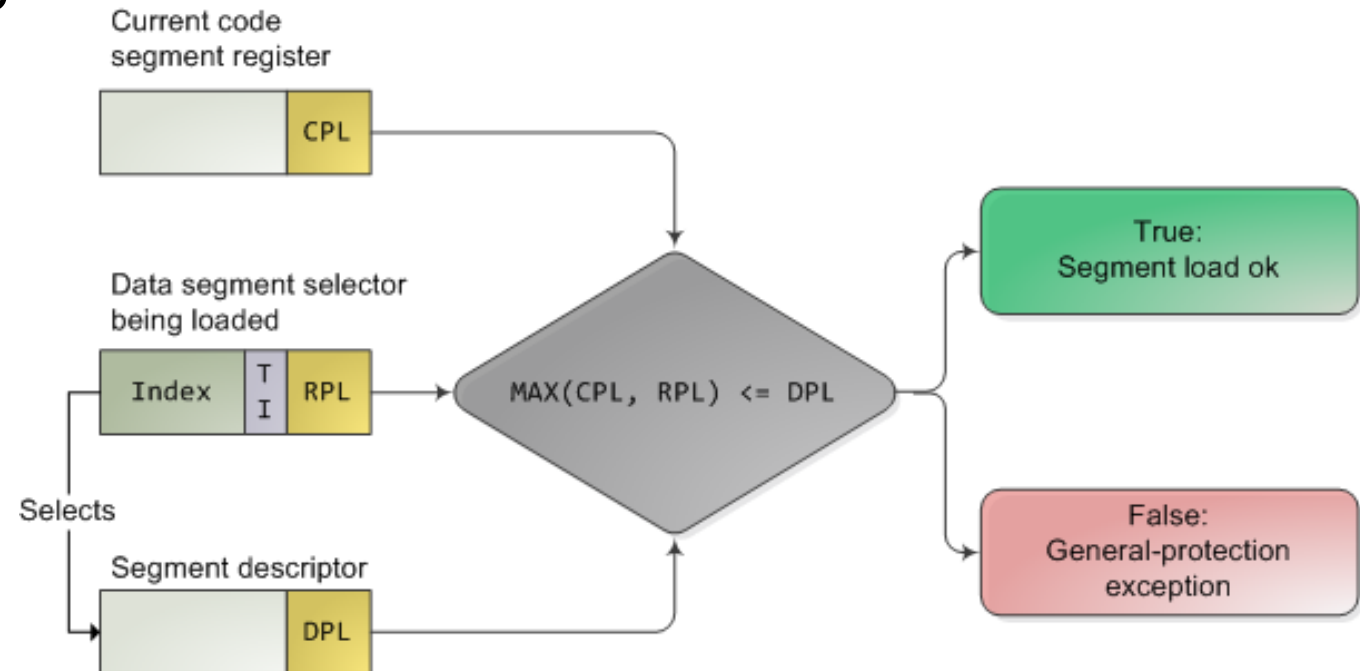
180386 — RESTRINGINDO ACESSO AOS DADOS

Ao endereçar um dado em memória, o programa deve carregar o seletor de um segmento de dados em um registrador de segmento de dados (DS, ES, FS, GS, SS)

O processador avalia automaticamente o acesso ao segmento de dados comparando os níveis de privilégio

- Isto ocorre durante o carregamento do seletor do segmento alvo

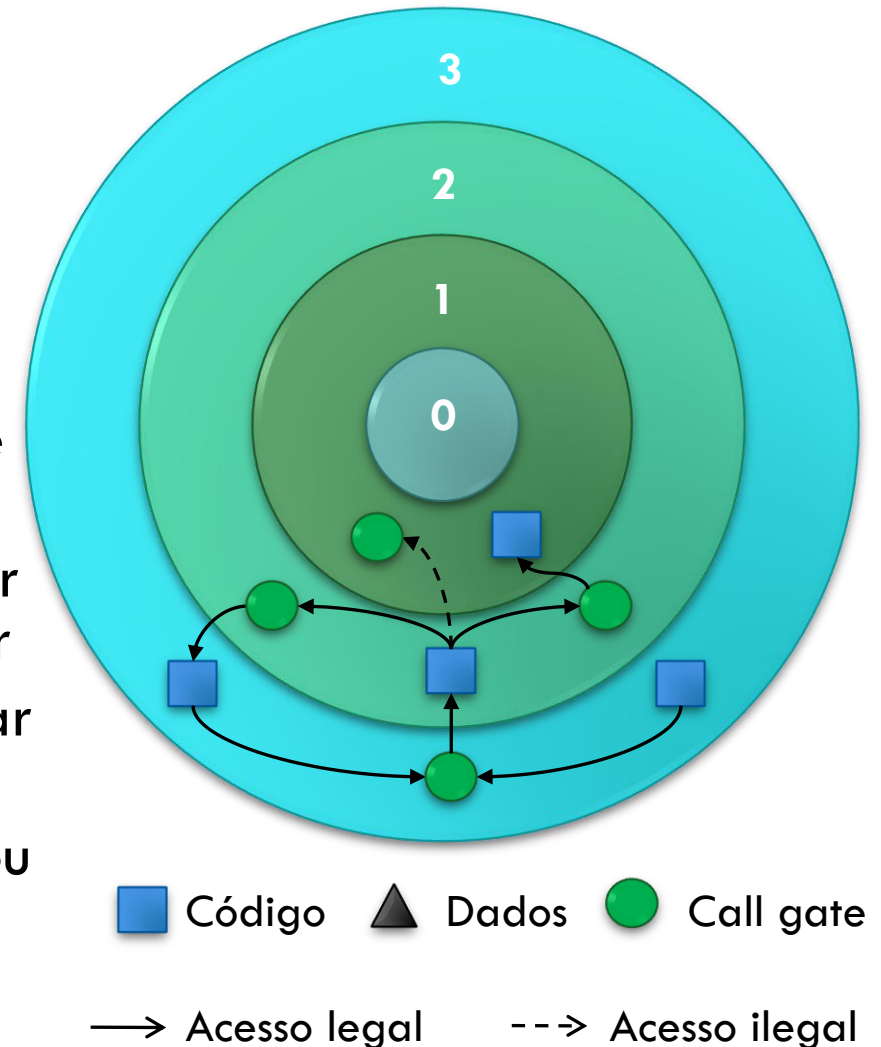
3 diferentes níveis de privilégio entram neste tipo de checagem: CPL, RPL, DPL



180386 — RESTRINGINDO ACESSO AO CÓDIGO

Call gates (portas de chamada) facilitam a transferência do controle de um programa entre diferentes níveis de privilégio

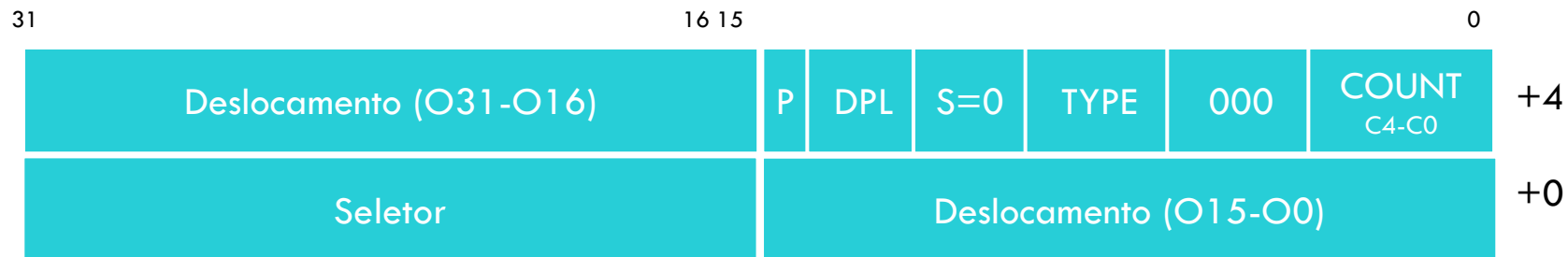
- Possuem um descritor próprio que apontam para um procedimento em um segmento de código que possui um nível de código diferente do nível daquele segmento
- Instrução CALL podem utilizar *call gates* para acessar segmentos de código com nível de privilégio superior
- Acessível também pela instrução JMP, mas sem alterar para nível de privilégio superior
- Os descritores das portas são preparados na GDT ou na LDT pelo SO



180386 — RESTRINGINDO ACESSO AO CÓDIGO

Portas de Chamada

- TYPE = 1100 (define que é uma porta de chamada – call gate)
- S=0 pois é um descritor do sistema
- O campo COUNT define quantas palavras duplas (até 31) devem ser copiadas da pilha do processo que fez a chamada para a nova pilha



I80386 — RESTRINGINDO ACESSO AO CÓDIGO

4 níveis de privilégio são usados para checar a validade da transferência de controle de um programa através de uma *call gate*

- CPL do seletor CS
- RPL do seletor do *call gate*
- DPL do descritor do *call gate*
- DPL do descritor de segmento do segmento de código destino

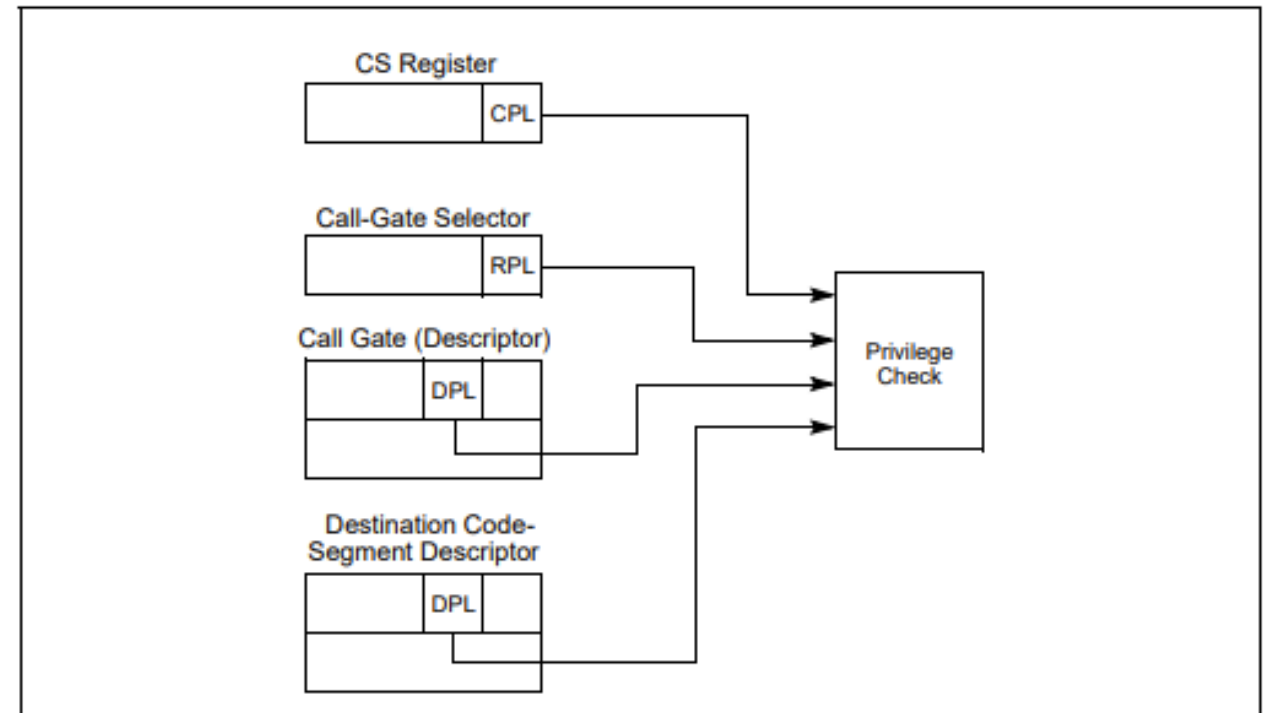
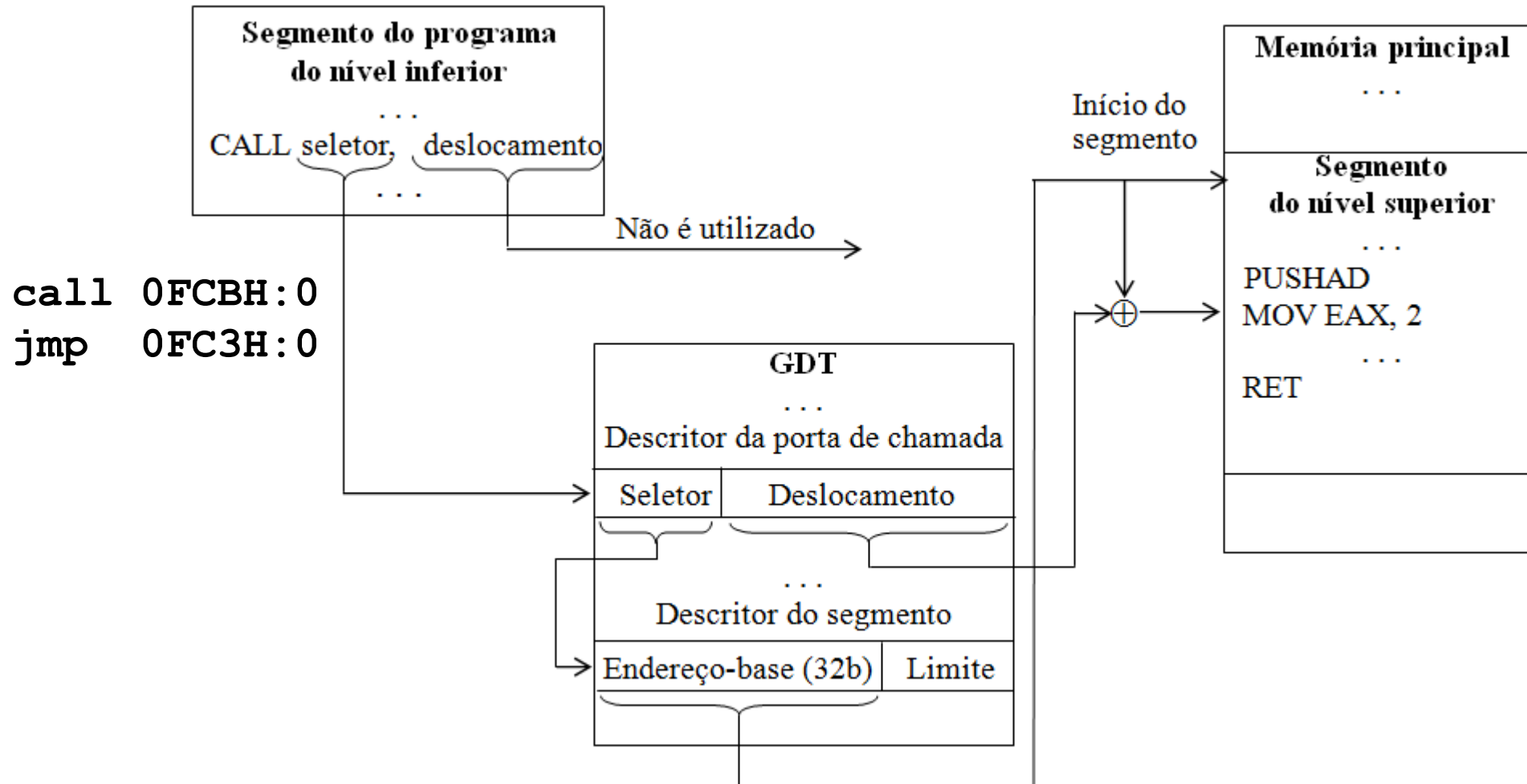


Figure 5-11. Privilege Check for Control Transfer with Call Gate

[Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3A: System Programming Guide, Part 1](#)

I80386 – RESTRINGINDO ACESSO AO CÓDIGO



180386 — RESTRINGINDO ACESSO AO CÓDIGO

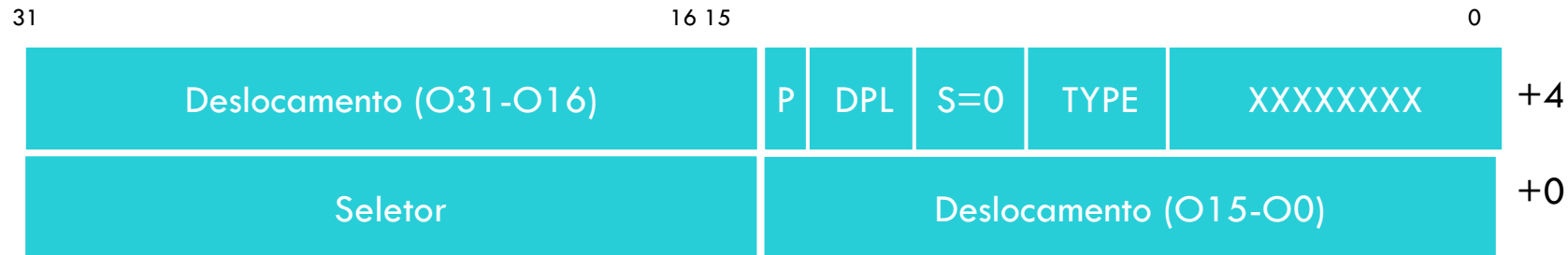
Além da porta de chamada, existem outros três tipos de portas

- Interrupção (interrupt gate): chamada pela instrução INT ou por interrupção de hardware
- Armadilha (trap gate): chamada pela instrução INT ou por interrupção de hardware
- Tarefa (task gate): chamada pela instrução JMP, CALL, e INT ou por interrupção de hardware. Utilizados para multi-tarefa

180386 — RESTRINGINDO ACESSO AO CÓDIGO

Portas de Interrupção

- A IDT no modo protegido possui descritores especiais que realizam o acesso a sub-rotinas de tratamento
- Tipos (IF=1/0)
 - Interrupt gate (TYPE = 1110)
 - Trap gate (TYPE = 1111)



180386 — RESTRINGINDO ACESSO AO CÓDIGO

Os descritores das porta de interrupção / armadilha estão armazenadas em uma tabela de descritores de interrupção (IDT — *Interrupt Descriptor Table*)

- Única para o sistema
- Contém até 256 descritores (task-gate, interrupt-gate, trap-gate) (tamanho máximo de 2KB)

IDTR (registrador de 48 bits) contém o **endereço linear** da IDT

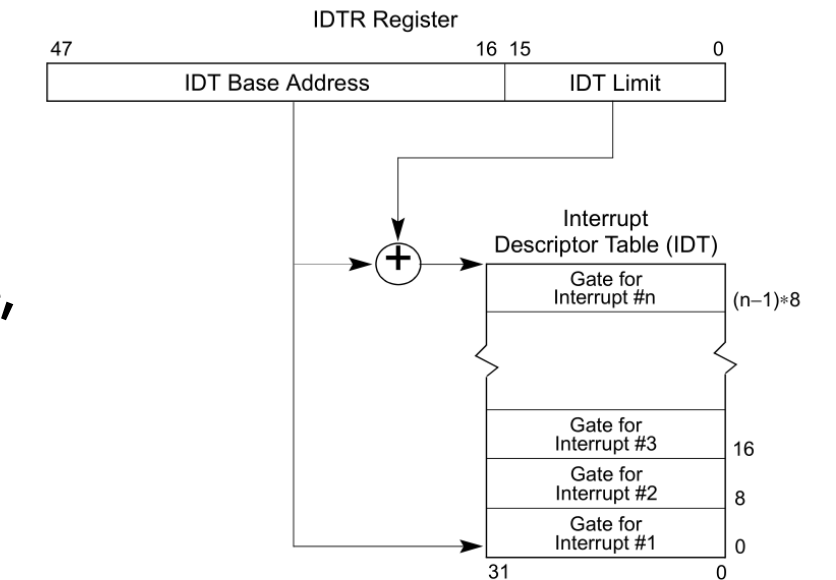


Figure 6-1. Relationship of the IDTR and IDT



I80386 — MECANISMO DE PROTEÇÃO

